

Trabajo De Base de Datos Entregable-1

Isabel Sofia Celedon Martinez

Jose Jorge Gonzalez Oñate

Imer Jose Iguaran Camargo

Juan David López Nuñez

Erick Jose Pinto carrillo

Ángel Gabriel Vargas Garcí

Asesor

Yair Alfredo Vargas Delgado

Universidad Popular Del Cesar UPC

Facultad de Ingenierías y Tecnológicas

2026

Contenido

1.PLANTEAMIENTO CRÍTICO DEL PROBLEMA	4
1.1 Descripción del problema	4
1.2. Problema central	5
1.3. Problemática desde la programación	5
2.Preguntas del negocio	5
2.1.Requisitos y supuestos principales	6
2.2.Justificación y conclusión	6
3.Modelo EER	7
.....	7
4.Esquema relacional	8
4.1El modelo conceptual de Sabor & Stock	8
4.2.Claves Primarias (PK)	9
4.3.Claves foráneas (FK)	9
4.4.Políticas de borrado (ON DELETE)	10
5.Diccionario De Datos De Uso Inicial	11
5.1. Tabla: unidad medida	12
5.2. Tabla: categoria	12
5.3. Tabla: proveedor	13
5.4. Tabla: ingrediente	13
5.5. Tabla: producto	14

5.6. Tabla: Receta	15
5.7. Tabla: Compra	15
5.8. Tabla: Detalle compra.....	16
5.9. Tabla: Movimiento Inventario	16

1. PLANTEAMIENTO CRÍTICO DEL PROBLEMA

Sistema de gestión de información para el inventario de un restaurante

Proyecto Integrador — Base de Datos I

Caso de estudio: Restaurante Sabor & Stock

Base de datos relacional y programación con PostgreSQL

1.1 Descripción del problema

Los restaurantes manejan diariamente información sobre ingredientes, productos, proveedores, compras y movimientos de inventario. Cuando estos datos se registran en hojas de cálculo, documentos o archivos independientes, se dificulta mantenerlos organizados, actualizados y disponibles. Esto puede ocasionar pérdidas, duplicidad de información y errores en el control de existencias.

Para este proyecto se plantea el caso del Restaurante Sabor & Stock, que necesita controlar los ingredientes utilizados para preparar sus productos, las compras realizadas a proveedores y las entradas y salidas del inventario. El problema principal es la falta de centralización y relación de los datos, lo que dificulta conocer con precisión qué existe, qué debe comprarse y cómo se ha utilizado cada insumo.

Desde el punto de vista de Bases de Datos, es necesario representar correctamente las relaciones entre proveedores, ingredientes, productos, recetas, compras y movimientos de inventario. Por ejemplo, un proveedor puede realizar varias compras; una compra puede contener varios ingredientes; y un producto puede requerir varios ingredientes. Una estructura relacional permitirá reducir duplicidades, inconsistencias y pérdida de información.

1.2. Problema central

El problema central es la deficiente administración y control de la información del inventario del restaurante debido al manejo disperso y manual de los datos.

Entre los principales problemas se encuentran:

- Dificultad para conocer el stock real de cada ingrediente.
- Riesgo de registrar información duplicada o inconsistente.
- Falta de control sobre entradas, salidas y ajustes de inventario.
- Dificultad para identificar ingredientes con stock bajo.
- Complicaciones para consultar compras, proveedores y costos.
- Riesgo de registrar cantidades o movimientos inválidos.

1.3. Problemática desde la programación

La solución no debe limitarse a almacenar registros; también debe controlar reglas de negocio. Por ejemplo, no debería ser posible registrar stock negativo, cantidades menores o iguales a cero, códigos duplicados o una salida superior a la existencia disponible.

PostgreSQL permitirá aplicar PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE y CHECK para garantizar la integridad de los datos. Además, funciones y triggers podrán automatizar operaciones como la actualización del stock y el cálculo de totales de compra. La aplicación permitirá registrar y consultar la información, mientras la base de datos se encargará de relacionarla, validarla y mantenerla consistente.

2.Preguntas del negocio

- ¿Qué ingredientes están disponibles actualmente?
- ¿Qué ingredientes tienen stock bajo?
- ¿Qué proveedores suministran cada ingrediente?

- ¿Cuánto se ha comprado durante un período?
- ¿Qué ingredientes se utilizan con mayor frecuencia?
- ¿Qué productos utilizan un determinado ingrediente?
- ¿Cuánto cuesta producir un producto según su receta?
- ¿Qué movimientos de inventario se realizaron en un período?
- ¿Qué compras tienen un costo superior al promedio?

2.1.Requisitos y supuestos principales

El sistema deberá manejar proveedores, categorías, unidades de medida, ingredientes, productos, recetas, compras, detalles de compra y movimientos de inventario. Cada ingrediente tendrá un código único, unidad de medida, stock y stock mínimo. Los productos pertenecerán a una categoría y podrán utilizar varios ingredientes.

Se asume que un proveedor puede realizar varias compras, una compra puede contener varios ingredientes y un ingrediente puede participar en diferentes compras y recetas. Las cantidades deberán ser mayores que cero, los precios no podrán ser negativos y una salida de inventario no deberá permitir que el stock quede por debajo de cero. Los precios aplicados en las compras deberán conservarse históricamente.

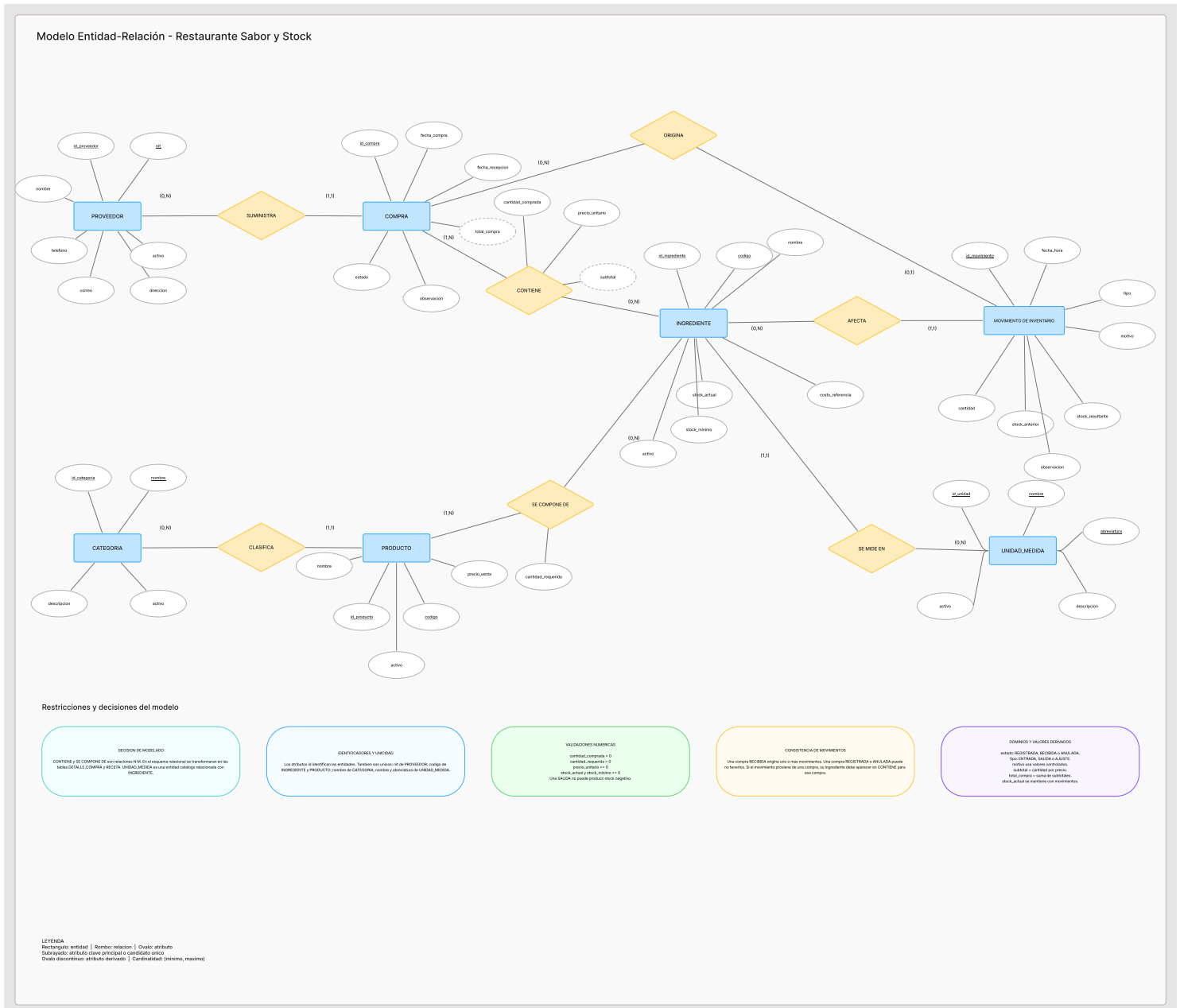
2.2.Justificación y conclusión

El proyecto permite aplicar conceptos fundamentales de Base de Datos I y programación sobre una situación real: la administración del inventario de un restaurante. Se trabajarán entidades, relaciones, cardinalidades, claves primarias y foráneas, restricciones, normalización, consultas SQL, vistas, funciones, triggers, transacciones, roles y mecanismos de respaldo.

En conclusión, una base de datos relacional permitirá transformar un manejo disperso del inventario en una estructura organizada, donde la información pueda consultarse y controlarse de

manera confiable. La solución no solo almacenará datos, sino que aplicará reglas de negocio y generará información útil para controlar existencias, compras y abastecimiento del restaurante.

3.Modelo EER



4. Esquema relacional

4.1 El modelo conceptual de Sabor & Stock

- Proveedor
- Categoría
- UnidadMedida
- Ingrediente
- Producto
- Receta
- Compra
- DetalleCompra
- MovimientoInventario

Cada entidad se traduce en una tabla; las relaciones 1:N se resuelven agregando la clave foránea en la tabla del lado "muchos", y las relaciones N:M (Producto–Ingrediente y Compra–Ingrediente) se resuelven creando una tabla intermedia con clave primaria compuesta.

Relación del EER	Cardinalidad	Mapeo relacional
Proveedor → Compra	1:N	FK id_proveedor en compra
UnidadMedida → Ingrediente	1:N	FK id_unidad en ingrediente
Categoría → Producto	1:N	FK id_categoria en producto
Ingrediente → MovimientoInventario	1:N	FK id_ingrediente en movimiento_inventario
Producto → Ingrediente	N:M	Tabla intermedia receta (PK compuesta)

Ingrediente→ MovimientoInventario	N:M	Tabla intermedia detalle_compra (PK compuesta)
--------------------------------------	-----	--

4.2. Claves Primarias (PK)

Cada tabla tiene un identificador único que garantiza que ninguna fila se repita. En las seis tablas base se usa una clave primaria simple autonumérica (SERIAL); en las tablas de detalle (receta y detalle_compra) se usa una clave primaria compuesta, formada por las dos claves foráneas que participan en la relación N:M.

- **Simples:** id_unidad, id_categoria, id_proveedor, id_ingrediente, id_producto, id_compra, id_movimiento.
- **Compuestas:**
 - (id_producto, id_ingrediente) en receta.
 - (id_compra, id_ingrediente) en detalle_compra.

4.3. Claves foráneas (FK)

Toda clave foránea apunta a la clave primaria de otra tabla y garantiza que no puedan existir referencias a registros inexistentes (**integridad referencial**). En este esquema todas las FK son NOT NULL, es decir, la relación es obligatoria: una compra siempre requiere un proveedor, un ingrediente siempre requiere una unidad de medida, etc.

Tabla (hijo)	Columna FK	Referencia (padre)	Obligatoria
ingrediente	id_unidad	unidad_medida(id_unidad)	Sí (NOT NULL)
producto	id_categoria	categoria(id_categoria)	Sí (NOT NULL)
compra	id_proveedor	proveedor(id_proveedor)	Sí (NOT NULL)
detalle_compra	id_compra	compra(id_compra)	Sí (NOT NULL)
detalle_compra	id_ingredient	ingrediente(id_ingredient)	Sí (NOT NULL)
receta	id_producto	producto(id_producto)	Sí (NOT NULL)
receta	id_ingredient	ingrediente(id_ingredient)	Sí (NOT NULL)
movimiento_inventario	id_ingredient	ingrediente(id_ingredient)	Sí (NOT NULL)

4.4. Políticas de borrado (ON DELETE)

Definir una FK no basta: también hay que decidir qué ocurre con las filas hijas cuando se intenta borrar la fila padre que referencian. Esa decisión se declara con ON DELETE y se conoce como *política o acción referencial*.

Política adoptada en este esquema

Sabor & Stock aplica ON DELETE RESTRICT en todas sus relaciones.

La razón de negocio es que el restaurante necesita conservar el historial completo de compras, recetas y movimientos de inventario para auditoría, costeo y trazabilidad: ningún proveedor, ingrediente, categoría, unidad de medida, producto o compra debe poder eliminarse mientras existan registros que dependan de él.

Las bajas reales se resuelven con *desactivación lógica* (por ejemplo, un futuro atributo activo en catálogos), nunca con DELETE físico sobre un padre referenciado.

Relación (padre → hijo)	Política	Efecto al intentar DELETE del padre
unidad_medida → ingrediente	RESTRICT	Bloquea el borrado si existen ingredientes con esa unidad.
categoria → producto	RESTRICT	Bloquea el borrado si existen productos en esa categoría.
proveedor → compra	RESTRICT	Bloquea el borrado si el proveedor tiene compras registradas.
ingrediente → receta	RESTRICT	Bloquea el borrado si el ingrediente se usa en alguna receta.
ingrediente → detalle_compra	RESTRICT	Bloquea el borrado si el ingrediente aparece en alguna compra.
ingrediente → movimiento_inventario	RESTRICT	Bloquea el borrado si el ingrediente tiene movimientos registrados.
producto → receta	RESTRICT	Bloquea el borrado si el producto tiene una receta definida.
compra → detalle_compra	RESTRICT	Bloquea el borrado si la compra tiene líneas de detalle.

5.Diccionario De Datos De Uso Inicial

Sistema de Gestión de Inventario — Restaurante Sabor & Stock

5.1. Tabla: unidad medida

Campo	Tipo de dato	Restricción	Clave	Descripción
id_unidad	SERIAL	NOT NULL	PK	Identificador único de la unidad de medida.
nombre	VARCHAR(50)	NOT NULL, UNIQUE	UQ	Nombre de la unidad de medida.
abreviatura	VARCHAR(10)	NOT NULL, UNIQUE	UQ	Abreviatura utilizada para representar la unidad de medida.
descripcion	VARCHAR(150)	—	—	Descripción adicional de la unidad de medida.

5.2. Tabla: categoria

Campo	Tipo de dato	Restricción	Clave	Descripción
id_categoria	SERIAL	NOT NULL	PK	Identificador único de la categoría.
nombre	VARCHAR(80)	NOT NULL, UNIQUE	UQ	Nombre de la categoría a la que pertenecen los productos.
descripcion	VARCHAR(200)	—	—	Información adicional sobre la categoría.

5.3. Tabla: proveedor

Campo	Tipo de dato	Restricción	Clave	Descripción
id_proveedor	SERIAL	NOT NULL	PK	Identificador único del proveedor.
documento	VARCHAR(30)	NOT NULL, UNIQUE	UQ	Número de documento que identifica al proveedor.
nombre	VARCHAR(120)	NOT NULL	—	Nombre del proveedor.
telefono	VARCHAR(30)	—	—	Número telefónico de contacto del proveedor.
correo	VARCHAR(120)	—	—	Correo electrónico de contacto del proveedor.
direccion	VARCHAR(180)	—	—	Dirección registrada del proveedor.

5.4. Tabla: ingrediente

Campo	Tipo de dato	Restricción	Clave	Descripción
id_ingrediente	SERIAL	NOT NULL	PK	Identificador único del ingrediente.
codigo	VARCHAR(30)	NOT NULL, UNIQUE	UQ	Código único utilizado para identificar el ingrediente.

nombre	VARCHAR(120)	NOT NULL	—	Nombre del ingrediente.
id_unidad	INT	NOT NULL	FK	Identificador de la unidad de medida utilizada por el ingrediente.
stock	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0 , DEFAULT 0	—	Cantidad disponible del ingrediente en inventario.
stock_minimo	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0 , DEFAULT 0	—	Cantidad mínima de existencia establecida para el ingrediente.
precio_referencia	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0 , DEFAULT 0	—	Precio de referencia utilizado para el ingrediente.

5.5. Tabla: producto

Campo	Tipo de dato	Restricción	Clave	Descripción
id_producto	SERIAL	NOT NULL	PK	Identificador único del producto.
codigo	VARCHAR(30)	NOT NULL, UNIQUE	UQ	Código único utilizado para identificar el producto.
nombre	VARCHAR(120)	NOT NULL	—	Nombre del producto.
id_categoria	INT	NOT NULL	FK	Identificador de la categoría a la que pertenece el producto.
precio_venta	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0	—	Precio establecido para la venta del producto.

5.6. Tabla: Receta

Campo	Tipo de dato	Restricción	Clave	Descripción
id_producto	INT	NOT NULL	PK / FK	Identificador del producto que utiliza la receta.
id_ingredient	INT	NOT NULL	PK / FK	Identificador del ingrediente utilizado en la receta.
cantidad	NUMERIC(12,3)	NOT NULL, CHECK > 0	—	Cantidad del ingrediente necesaria para la elaboración del producto.

5.7. Tabla: Compra

Campo	Tipo de dato	Restricción	Clave	Descripción
id_compra	SERIAL	NOT NULL	PK	Identificador único de la compra.
id_proveedor	INT	NOT NULL	FK	Identificador del proveedor al que se realiza la compra.
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	Fecha y hora en que se registra la compra.
estado	VARCHAR(20)	NOT NULL, CHECK IN (REGISTRADA, RECIBIDA, ANULADA), DEFAULT 'REGISTRADA'	—	Estado actual de la compra.

total	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0 , DEFAULT 0	—	Valor total de la compra.
-------	---------------	---	---	---------------------------

5.8. Tabla: Detalle compra

Campo	Tipo de dato	Restricción	Clave	Descripción
id_compra	INT	NOT NULL	PK / FK	Identificador de la compra a la que pertenece el detalle.
id_ingrediente	INT	NOT NULL	PK / FK	Identificador del ingrediente adquirido.
cantidad	NUMERIC(12,3)	NOT NULL, CHECK > 0	—	Cantidad del ingrediente incluida en la compra.
precio_aplicado	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0	—	Precio aplicado al ingrediente en esa compra.

5.9. Tabla: Movimiento Inventario

Campo	Tipo de dato	Restricción	Clave	Descripción
id_movimiento	SERIAL	NOT NULL	PK	Identificador único del movimiento de inventario.
id_ingrediente	INT	NOT NULL	FK	Identificador del ingrediente relacionado con el movimiento.
tipo	VARCHAR(10)	NOT NULL, CHECK IN (ENTRADA, SALIDA, AJUSTE)	—	Tipo de movimiento realizado sobre el inventario.

cantidad	NUMERIC(12,3)	NOT NULL, CHECK > 0	—	Cantidad de unidades involucradas en el movimiento.
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	Fecha y hora en que se registra el movimiento.
motivo	VARCHAR(200)	NOT NULL	—	Razón por la cual se realiza el movimiento.

Resumen del diccionario

El diccionario de datos de uso inicial está compuesto por 9 tablas:

- unidad_medida
- categoria
- proveedor
- ingrediente
- producto
- receta
- compra
- detalle_compra
- movimiento_inventario